30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 10

Testing Strategies — Part 2: Integration, UI, Screenshot & Performance Tests



Esmaeeil Moradi
@esmaeeil-moradi700

Why UI and integration tests matter in 2026

Unit tests (Day 9) prove the pieces work. This part proves they work **together, on screen, on real devices** — without an hour-long, flaky suite:

- Compose made UI tests **semantic**: find nodes by meaning, not by view ID
- Robolectric and screenshot tests moved many UI tests **off the device**, onto the JVM
- Visual regressions are caught by **screenshot diffs**, not by someone clicking through QA
- Performance is testable: **Macrobenchmark** measures startup and jank as code
- Gradle Managed Devices make device runs reproducible (CI wiring is Day 23)

Level	Questions
Junior	Compose UI tests · Espresso · How tests synchronize
Mid	Testing a screen · Robolectric · Integration tests · Leaving your app
Senior	Screenshot tests · Flaky tests · Macrobenchmark as tests
Architect	Gradle Managed Devices · Sharding & isolation · UI tests at scale



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: where each test runs

JVM	JVM + Robolectric	Device / GMD
logic, VMs, repositories	screens, DAOs, screenshots	E2E journeys, system UI, benchmarks
fastest	fast	slow, flakiest

- Put each test in the **cheapest column** that can still catch the bug
- Devices are for what only a device can prove: real rendering, system UI, performance
- In every column, tests are **hermetic**: no real backend, no state shared between tests

THE ONE RULE BEHIND EVERY ANSWER TODAY

A UI test should fail for one reason only: the UI is wrong. Not the network, not the timing, not the test that ran before it.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 10



Junior

Compose UI tests · Espresso · How tests synchronize



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. A Compose UI test

```
@get:Rule val compose = createComposeRule()

@Test fun loginButton_enabledOnlyWithValidInput() {
    val vm = LoginViewModel(FakeAuthRepo())
    compose.setContent { AppTheme { LoginScreen(vm) } }
    val login = compose.onNodeWithText("Log in")
    login.assertIsNotEnabled()

    compose.onNodeWithTag("email").performTextInput("a@b.co")
    compose.onNodeWithTag("password").performTextInput("s3cret")

    login.assertIsEnabled()
}
```

- Compose tests query the **semantics tree** — the same tree accessibility services read — not views
- `createComposeRule()` hosts content in an empty activity; `createAndroidComposeRule<MainActivity>()` launches a real one



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. Find · act · assert

Step	API
Find	`onNodeWithText`, `onNodeWithTag`, `onNodeWithContentDescription`, `onNode(hasText("Pay") and hasClickAction())`
Act	`performClick()`, `performTextInput()`, `performScrollTo()`, `performTouchInput { swipeUp() }`
Assert	`assertIsDisplayed()`, `assertIsEnabled()`, `assertTextEquals()`, `assertDoesNotExist()`
Debug	`onRoot().printToLog("TAG")` prints the semantics tree

INTERVIEW TRAP

"Compose UI tests need a device." — The same `createComposeRule()` test runs on the JVM with Robolectric (Q5). Where it runs is a Gradle source-set choice, not a different API.

SETUP DETAIL

`createComposeRule()` needs `debugImplementation("androidx.compose.ui:ui-test-manifest")` — it declares the empty activity the rule launches. Without it, the rule can't start.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Espresso for View-based screens

```
@RunWith(AndroidJUnit4::class)
class CheckoutActivityTest {
    @get:Rule
    val rule = activityScenarioRule<CheckoutActivity>()

    @Test fun shortCardNumber_showsError() {
        onView(withId(R.id.card_number))
            .perform(typeText("4242"), closeSoftKeyboard())
        onView(withId(R.id.pay)).perform(click())

        onView(withText(R.string.error_card_invalid))
            .check(matches(isDisplayed()))
    }
}
```

- `onView(matcher)` finds · `perform(action)` acts · `check(assertion)` verifies
- Lists: `RecyclerViewActions` (espresso-contrib); `onData()` only for `AdapterView`
- Hybrid screens: Espresso and Compose APIs work in the same test with `createAndroidComposeRule`



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. What interviewers check

INTERVIEW TRAP

`Thread.sleep(2000)` "to wait for the data". Too slow on fast devices, too short on slow ones — the classic source of flaky tests. Synchronize on conditions instead (Q3).

INTERVIEW TRAP

"Espresso is obsolete now that we have Compose." — Most production apps still have View screens, and Espresso is still the tool for them — including Views hosted inside Compose and Compose inside Views.

FOLLOW-UP

"*`ActivityTestRule` or `ActivityScenario`?*" — `ActivityTestRule` is deprecated. `ActivityScenario` (or `ActivityScenarioRule`) drives the lifecycle explicitly: `moveToState()`, `recreate()`, `onActivity { }`.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. How tests synchronize — and what they miss

	Waits for	Does not wait for
Espresso	Main-thread message queue idle; registered `IdlingResource`s	Your coroutines on background dispatchers; running animations
Compose test rule	Pending recompositions, layout, draw; animations on its `mainClock`; registered idling resources	Background work that isn't registered or faked

INTERVIEW TRAP

"Espresso automatically waits for animations and data loading." — It waits for the **main thread** and registered idling resources. A request on `Dispatchers.IO` is invisible to it, and system animations cause flakiness — turn them off (`testOptions.animationsDisabled = true`).



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Waiting the right way

```
// Best: hermetic fakes that answer at once — no waiting

// Compose: wait for a condition, never for a duration
compose.waitUntil(timeoutMillis = 5_000) {
    compose.onAllNodesWithText("Paid")
        .fetchSemanticsNodes().isNotEmpty()
}

// Compose: drive animations by hand
compose.mainClock.autoAdvance = false
compose.onNodeWithText("Details").performClick()
compose.mainClock.advanceTimeBy(300)
compose.onNodeWithTag("details_panel").assertIsDisplayed()

// Espresso: expose real async work as an IdlingResource
IdlingRegistry.getInstance().register(idlingResource)
```

INTERVIEW TRAP

Sprinkling `CountingIdlingResource.increment()` through production code couples it to tests. Prefer injected fakes or test dispatchers — then there's nothing to wait for.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 10



Mid-Level

Testing a screen · Robolectric · Integration tests · Leaving your app



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Testing a screen: state in, events out

```
@Test fun errorState_showsMessage_andReportsRetry() {
    var retries = 0
    compose.setContent {
        AppTheme {
            FeedScreen(
                state = FeedUiState(userMessage = "Offline"),
                onRetry = { retries++ },
            )
        }
    }
    compose.onNodeWithText("Offline").assertIsDisplayed()
    compose.onNodeWithText("Retry").performClick()
    assertEquals(1, retries)
}
```

- The **stateless screen** from Day 3 pays off: every state is one `setContent` away — no ViewModel, no network
- Test each state: loading, empty, error, content, long text, large font



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Semantics: find nodes like a user

- Prefer what the user perceives: text, content description, role. Use `testTag` when there's no stable text.
- A `Button` **merges** its children's semantics — pass `useUnmergedTree = true` to reach a child node.
- Compose tags are not resource IDs. Set `Modifier.semantics { testTagsAsResourceId = true }` at the root so UI Automator and Macrobenchmark can use `By.res("…")`.
- Automated accessibility checks: Espresso's `AccessibilityChecks.enable()` runs the Accessibility Test Framework on every action.

INTERVIEW TRAP

"Run Accessibility Scanner in UI tests." — Accessibility Scanner is a **manual** app. Automated checks use the Accessibility Test Framework (ATF).

INTERVIEW TRAP

Hard-coded English strings in assertions break in other locales. Read them from resources (`compose.activity.getString(…)` with `createAndroidComposeRule`) or use tags.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Robolectric: Android tests on the JVM

```
// src/test — no emulator
@RunWith(AndroidJUnit4::class)
@Config(qualifiers = "w360dp-h640dp-xhdpi")
class SettingsScreenTest {
    @get:Rule val compose = createComposeRule()

    @Test fun darkModeSwitch_reportsChange() {
        var dark = false
        compose.setContent {
            SettingsScreen(dark = false, onDarkChange = {
                dark = it
            })
        }
        compose.onNodeWithTag("dark_switch").performClick()
        assertTrue(dark)
    }
}
```

- Same APIs as device tests: `AndroidJUnit4`, `ActivityScenario`, the Compose rule — the runner decides where they run
- Needs `testOptions.unitTests.isIncludeAndroidResources = true`



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Robolectric vs a device

	Robolectric	Device / emulator
Speed	JVM, parallel Gradle test forks	Install, boot, run
Fidelity	Real framework code; simulated system services	Everything real
Good for	Screens, resources, DAOs, screenshots (`@GraphicsMode(NATIVE)`)	System UI, vendor quirks, GPU rendering, performance

INTERVIEW TRAP

"Robolectric mocks Android, so the results are fake." — It runs the **real Android framework code** for the configured SDK and replaces only the native and system parts with shadows. What it can't prove — real devices, other processes, performance — stays in a smaller device suite.

FOLLOW-UP

"Does it work with Compose?" — Yes. Google's Now in Android sample runs its Compose screenshot tests on Robolectric.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Integration tests: real Room, in memory

```
@RunWith(AndroidJUnit4::class)
class ArticleDaoTest {
    private lateinit var db: AppDatabase

    @Before fun setUp() {
        db = Room.inMemoryDatabaseBuilder(
            ApplicationProvider.getApplicationContext(),
            AppDatabase::class.java,
        ).build()
    }

    @After fun tearDown() = db.close()

    @Test fun upsert_replacesExistingRow() = runTest {
        val dao = db.articleDao()
        dao.upsertAll(listOf(article(id = 1, title = "Old")))
        dao.upsertAll(listOf(article(id = 1, title = "New")))
        val rows = dao.observeAll().first()
        assertEquals("New", rows.single().title)
    }
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. The other boundaries

Boundary	Integration test
Room migrations	`MigrationTestHelper` + exported schemas — every migration path
Network	MockWebServer (OkHttp) or Ktor `MockEngine`: real client + serializer, canned responses — Day 11
DI graph	Hilt `@TestInstallIn` / `@UninstallModules` swaps bindings app-wide — Day 7
Navigation	`TestNavHostController` — assert on the back stack

INTERVIEW TRAP

"Our in-memory DB tests prove migrations work." — An in-memory database is created fresh from the **current** schema; no migration ever runs. Test migrations with `MigrationTestHelper`.

INTERVIEW TRAP

Integration tests against the real staging backend: slow, shared data, down at the worst moment. Keep them hermetic; check the API contract separately.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Leaving your app: intents, permissions, system UI

```
@get:Rule val intents = IntentsRule()

@Test fun takePhoto_showsPreview() {
    val result = Instrumentation.ActivityResult(
        Activity.RESULT_OK, Intent(),
    )
    intending(hasAction(MediaStore.ACTION_IMAGE_CAPTURE))
        .respondWith(result)

    onView(withId(R.id.take_photo)).perform(click())
    onView(withId(R.id.preview)).check(matches(isDisplayed()))
}
```

- Espresso-Intents: `intended(…)` verifies outgoing intents, `intending(…)` stubs the result — no camera app needed
- `GrantPermissionRule.grant(Manifest.permission.CAMERA)` when the dialog isn't what you're testing
- UI Automator (`UiDevice`, `By.text(…)`) for notifications, permission dialogs, the launcher, other apps

INTERVIEW TRAP

"Espresso can tap the permission dialog." — It can't: the dialog belongs to another process. Espresso only sees your app; use UI Automator.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 10



Senior

Screenshot tests · Flaky tests · Macrobenchmark as tests



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Screenshot tests with Roborazzi

```
@RunWith(AndroidJUnit4::class)
@GraphicsMode(GraphicsMode.Mode.NATIVE)
@Config(qualifiers = RobolectricDeviceQualifiers.Pixel5)
class CheckoutScreenshotTest {
    @get:Rule val compose = createComposeRule()

    @Test fun checkout_errorState() {
        compose.setContent {
            AppTheme { CheckoutScreen(errorState, onPay = {}) }
        }
        compose.onRoot().captureRoboImage()
    }
}
```

```
./gradlew recordRoborazziDebug   # write golden images
./gradlew verifyRoborazziDebug   # fail on pixel differences
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Choosing a screenshot tool

Tool	Renders with	Notes
Roborazzi	Robolectric native graphics, JVM	Can capture after interactions; Compose and Views
Paparazzi	layoutlib (Android Studio's preview renderer), JVM	Fast; renders a composable or view — no activity, no interactions
Compose Preview Screenshot Testing	layoutlib, reuses `@Preview`s	Google's Gradle plugin; launched as experimental
Device screenshots	Real GPU	Highest fidelity, slowest, noisiest

INTERVIEW TRAP

Recording goldens on a Mac and verifying on Linux CI: fonts and anti-aliasing differ, so everything "fails". Record and verify in the **same environment**, and review image diffs in the pull request.

INTERVIEW TRAP

Screenshot tests don't test behavior — a pixel-perfect button that does nothing still passes. And raising the diff threshold until it's green hides real regressions.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Flaky tests: causes and fixes

Cause	Fix
Real network or backend	Hermetic fakes, MockWebServer
`sleep()` and timing guesses	Conditions, idling resources, test dispatchers
Animations	`animationsDisabled = true`
State leaking between tests (prefs, DB, singletons)	Test Orchestrator + `clearPackageData`, fresh in-memory DB
Order dependence	Each test sets up everything it needs
Time zone, locale, current date	Inject `Clock` and locale; fix them in tests
Overloaded emulators	GMD with ATD images, fewer parallel devices

A flaky test is a test that passes **and** fails on the same code. It destroys the one thing a suite exists for: trust in a red build.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Isolation and a flakiness policy

```
android {
    defaultConfig {
        testInstrumentationRunner =
            "androidx.test.runner.AndroidJUnitRunner"
        testInstrumentationRunnerArguments["clearPackageData"] =
            "true"
    }
    testOptions {
        execution = "ANDROIDX_TEST_ORCHESTRATOR"
        animationsDisabled = true
    }
}
dependencies {
    androidTestUtil(libs.androidx.test.orchestrator)
}
```

- **Orchestrator**: each test in its own instrumentation run — one crash fails one test, not the whole run
- **Policy**: detect (track flake rate per test) → quarantine with an owner → fix the root cause → bring it back

INTERVIEW TRAP

"Retry failed tests three times." — Retries turn red into green without fixing anything, and some "flaky" tests are real race conditions your users will hit. Retry to **measure** flakiness, not to hide it.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Macrobenchmark: performance as a test

```
@get:Rule val rule = MacrobenchmarkRule()

@Test fun coldStartup() = rule.measureRepeated(
    packageName = "com.example.app",
    metrics = listOf(StartupTimingMetric()),
    iterations = 10,
    startupMode = StartupMode.COLD,
) {
    pressHome()
    startActivityAndWait()
}

@Test fun feedScroll() = rule.measureRepeated(
    packageName = "com.example.app",
    metrics = listOf(FrameTimingMetric()),
    iterations = 5,
    setupBlock = { startActivityAndWait() },
) {
    device.findObject(By.res("feed")).fling(Direction.DOWN)
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Rules for meaningful numbers

- Lives in a separate `com.android.test` module and drives the app **from outside**, like a user
- Measure a release-like build: not debuggable, R8 on, `<profileable android:shell="true"/>`
- Run on a **physical device** — emulator numbers aren't representative, and the library reports an error on emulators unless you suppress it
- Compare `CompilationMode.None()` with the default to see what your Baseline Profile buys — Day 18

INTERVIEW TRAP

"Macrobenchmark is for Jetpack Compose." — It measures the whole app — Views or Compose — from outside. Measuring a function in-process is **Microbenchmark**.

INTERVIEW TRAP

"The benchmark fails when startup regresses." — It doesn't; it outputs results (JSON with min, median, max). Detecting regressions means comparing runs over time in CI — Day 28.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 10

→

Architect

Gradle Managed Devices · Sharding & isolation · UI tests at scale



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Gradle Managed Devices

```
android {
    testOptions {
        managedDevices {
            localDevices {
                create("pixel6api34") {
                    device = "Pixel 6"
                    apiLevel = 34
                    systemImageSource = "aosp-atd"
                }
            }
        }
    }
}
```

```
./gradlew pixel6api34DebugAndroidTest
```

- Gradle downloads the image, creates the emulator, runs the tests and shuts it down — the **same device** on every laptop and CI runner
- **Device groups** run one suite on phone, tablet and oldest supported API in one task



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Using GMD well

- **ATD images** (`aosp-atd`, `google-atd`): apps and services that tests rarely need are removed — less CPU and memory per emulator
- A test that relies on something ATD removed goes to a full `aosp` or `google` image
- The Firebase Test Lab plugin for GMD runs the same DSL on real devices in the cloud

INTERVIEW TRAP

"Emulators in CI are good enough for everything." — Not for performance numbers (Q10) or vendor-specific behavior. GMD makes emulator runs **reproducible**, not representative of every phone.

FOLLOW-UP

"How does this run on every pull request?" — Pipeline design, caching and which suites run when are CI/CD topics — Day 23.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Sharding: split the suite across devices

```
# AndroidJUnitRunner: run shard 0 of 4 on one device
adb shell am instrument -w -e numShards 4 -e shardIndex 0 \
  com.example.test/androidx.test.runner.AndroidJUnitRunner

# The same through Gradle
./gradlew connectedDebugAndroidTest \
  -Pandroid.testInstrumentationRunnerArguments.numShards=4 \
  -Pandroid.testInstrumentationRunnerArguments.shardIndex=0
```

```
# gradle.properties — GMD splits one suite over N emulators
android.experimental.androidTest.numManagedDeviceShards=4
```

INTERVIEW TRAP

"Sharding fixes a slow, flaky suite." — It fixes wall-clock time only. Shards that share accounts or backend data make flakiness worse. And the runner splits by test **count**, not duration — one slow class can dominate a shard; duration-based tools such as Flank balance better.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Isolation toolbox

Technique	Solves
Sharding (`numShards`, GMD shards, device farms)	Wall-clock time
Test Orchestrator + `clearPackageData`	One crash killing the run; state leaking between tests
Hermetic backend via DI	Backend outages, test data drift
Size filters (`@SmallTest`, `@LargeTest`, `-e size`)	Running the right subset at the right time
Unique test data per test	Shards colliding on shared accounts

A fast suite is a side effect of isolated tests — not of more machines.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. UI tests at scale: the robot pattern

```
class CheckoutRobot(private val rule: ComposeContentTestRule) {
    fun enterCard(number: String) = apply {
        rule.onNodeWithTag("card").performTextInput(number)
    }
    fun pay() = apply {
        rule.onNodeWithText("Pay").performClick()
    }
    fun assertReceiptShown() = apply {
        rule.onNodeWithTag("receipt").assertIsDisplayed()
    }
}

fun checkout(
    rule: ComposeContentTestRule,
    block: CheckoutRobot.() -> Unit,
) {
    CheckoutRobot(rule).block()
}

@Test fun validCard_showsReceipt() {
    checkout(compose) {
        enterCard("4242 4242 4242 4242")
        pay()
        assertReceiptShown()
    }
}
```

- Tests read as **user intent**; when the UI changes, you fix one robot, not forty tests



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. A UI test portfolio

Layer	What	Where	How many
Screen tests	Every state of every screen	JVM (Robolectric)	Many
Screenshot tests	Key screens × theme × font scale × RTL	JVM	Many, cheap
Integration	DAO, migrations, network, DI, navigation	JVM or device	Some
E2E journeys	Login, purchase, onboarding — hermetic backend	GMD / device farm	Few
Benchmarks	Startup, key scrolls	Physical device	A handful

INTERVIEW TRAP

"Every production bug gets an E2E test." — Add the test at the **lowest level** that reproduces the bug. E2E is for journeys, not for every regression.

SENIOR ANSWER IN ONE LINE

"Push every UI test to the cheapest environment that can still catch the bug, keep devices for what only a device can prove, and treat a flaky test as a bug — not as noise."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 10 · ARCHITECT

Day 10 — cheat sheet

- Compose tests query the **semantics tree**: find like a user, `testTag` as fallback.
- Espresso waits for the main thread and idling resources — **not** your background coroutines. Disable animations.
- Wait for conditions (`waitUntil`, idling resources), never `Thread.sleep()`.
- Test screens as state in, events out — stateless composables with fake state.
- Robolectric runs real framework code on the JVM; keep device tests for what it can't prove.
- In-memory Room for DAOs, `MigrationTestHelper` for migrations, MockWebServer for the network.
- System UI is another process: UI Automator or `GrantPermissionRule`, not Espresso.
- Screenshot tests catch visual regressions — record and verify in the same environment.
- Flaky tests: quarantine and fix; retries measure, they don't cure. Orchestrator + `clearPackageData` isolate.
- Macrobenchmark on a physical device and a release-like build; GMD + sharding for reproducible, faster device runs.

Day 11 is next → Network Layer, Part 1. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700